

ECE 309 Project 1: An LLM Mini-Harness in C via Vibe Coding

Deadline: Sep. 5, 2026

Project Overview & Goals

In this foundational project, you will build a minimal LLM agent harness in C. An agent harness acts as the bridge between an LLM and the operating system, managing inputs, context boundaries, tool execution, etc.

Instead of writing every line manually, you will utilize **vibe coding**—steering an AI assistant through clear, structured prompts to generate the C codebase. Do not panic if you feel that you don't write a line of code in this project. For this project, you are the architect, and the AI is your junior developer. Your job is to write a crystal-clear specification, feed it to an AI (like ChatGPT, Claude, or Gemini), and verify that the generated code actually works.

By completing this project, you will:

- **Understand** the architectural role of a harness in an LLM agent system.
- **Practice** Specification Driven Development (SDD) by writing rigorous technical requirements before instructing the AI to generate code.
- **Gain experience** managing and verifying AI-assisted coding workflows.

Implementation Requirements

Your harness must be written in standard C and compile successfully in a POSIX environment.

1. **Core Loop:** Implement a terminal-based loop that captures user input, passes it to a mock model function (which mimics an LLM), and outputs the simulated response. Note that the purpose of the mock model is that you don't need call the LLM APIs.
2. **Context Management:** Allocate and manage memory safely to store a minimal conversation history (e.g., the last 5 turns)
3. **Tool execution:** calls a tool to execute functions such as mathematical calculation that an LLM is not designed for.
4. **Vibe Coding Log:** Document the architectural rules and prompts you used to generate the C code, demonstrating your application of SDD principles.

5. **AI-Generated Testing:** Instruct your AI assistant to write a separate testing script that validates your harness's state management and checks for basic memory leaks.

Step-by-Step Guide

Phase 1: Environment Setup (VS Code)

Before writing any code, establish a standardized development environment to minimize system-specific compiler errors.

- **Install Visual Studio Code:** Download and install VS Code, then add the official **C/C++ Extension** by Microsoft from the extensions marketplace.
- **Configure the Compiler:**
 - *Mac users:* Install Xcode Command Line Tools by running `xcode-select --install` in the terminal.
 - *Windows users:* Install WSL (Windows Subsystem for Linux) and install GCC via the Ubuntu terminal using `sudo apt install gcc`.
- **Create a Workspace:** Open VS Code, create a new empty folder named `ece309_harness`, and open this folder in the editor to keep your project isolated.

Phase 2: Specification Driven Development (SDD) & The "Vibe"

In vibe coding, the AI requires a rigorous technical specification to produce functional C code rather than messy, over-complicated guesses. Do not just tell the AI to "write a chat app in C."

- **Define the State Machine:** Outline exactly how the program starts, how it loops, and what triggers a safe shutdown (e.g., typing "exit").
- **Set Hard Constraints:** Explicitly instruct the AI to avoid external libraries. Limit it strictly to standard C libraries like `<stdio.h>` and `<string.h>`.
- **Specify Input/Output:** Detail that the program must read strings from standard input (terminal), parse them, and print the mock model's text back.

The "Vibe" Prompt Template: Copy, modify, and paste a prompt similar to this into your AI assistant:

"I need to write a simple command-line program in C. I am a beginner, so please keep the code as simple as possible. Do not use external libraries, only standard ones like `<stdio.h>` and `<string.h>`. Here is the specification for the program:

1. It should run an infinite while loop that asks for user input using `fgets`.
2. If the user types 'exit', the loop should break and the program should end.

3. If the user types a sentence containing the word 'hello', the program should print a hardcoded greeting.
4. If the user types anything else, it should echo their input back to them.
5. Please add clear, line-by-line comments explaining what the code is doing."

Phase 3: Vibe Coding, Expected Output & Compilation

Feed your SDD rules into your chosen LLM and ask for a heavily commented, single-file C implementation. If your specification is good, the AI should generate something like the code below.

1. **Save the File:** In VS Code, create a new file named `harness.c` and paste your AI-generated code.
2. **Iterative Debugging & Compilation:** Open your terminal (Mac/Linux) or WSL (Windows) inside VS Code (Ctrl + ~ or Cmd + ~). Compile the code by running this exact command:

```
gcc harness.c -o harness
```

(If it prints nothing and goes to the next line, it worked! If it prints errors, copy those errors, paste them back into the AI, and ask it to fix the code).

3. **Run the Program:** Test it manually by typing `./harness`, testing your keywords ("hello", random gibberish), and finally "exit".

Phase 4: AI-Assisted Testing

Verify the harness works automatically to prepare for grading. Ask the AI to create a simple Bash script (`test.sh`) that pipes predefined text into your compiled program to verify its outputs deterministically.

AI Prompt Template for Testing:

"I have a compiled C program named `harness`. Write a very simple Bash script (for Linux/Mac) that automatically sends the word 'hello', followed by the word 'exit', into the program to test if it works."

The AI will generate a script (e.g., `test.sh`). Run it using `bash test.sh` to prove your harness works without human interaction.

Submission Guidelines

Version control is a critical engineering practice. You must host your project independently before submission.

1. **Initialize a GitHub Repository:** It must contain your source C files, headers, test scripts, and a descriptive README.md.
2. **Document the Vibe:** Save your exact prompts, iterations, and the AI's responses in a `vibe_coding_log.md` file and push it to your repository.
3. **Prepare Final Submission Files:** Create the following two files to submit for grading:
 - `github.txt`: A simple text file containing the direct URL to your GitHub repository.
 - `github.zip`: A compressed ZIP file containing your entire repository (source code, tests, logs, and README) as a backup.

Submit both `github.txt` and `github.zip` before the project deadline.